Test Environment, Containers & Test Data

Day 6 · Module 6 · 2 Hours

SDET / DevOps Engineer

Intermediate

40% Theory / 60% Hands-on

TOOLS

Docker / Docker Compose

Testcontainers (Java)

LEARNING OUTCOMES

What You Will Achieve Today

Containerise test execution using Docker

Write a Dockerfile for test execution, run automated tests inside containers, and wire Docker-based test jobs into a CI pipeline for consistent, reproducible results.

Manage test data safely across environments

Apply seed-script patterns, UUID-prefixed isolation, and environment-specific configuration to keep test data clean, reproducible, and compliant across dev, CI, and staging.

Provision and configure test environments reliably

Use Testcontainers for programmatic on-demand containers in integration tests, and manage environment-specific configuration via config files and CI env-vars without hardcoding.

Test Environment Provisioning Strategies

📊 Test environment instability is consistently cited as one of the most significant blockers in test automation. The core problem: tests that fail due to environment issues, not code defects, erode team confidence in CI pipelines.

Shared Persistent Environment

✓ PROS

- No spin-up time
- Simple setup

✕ CONS

- Data pollution between runs
- Configuration drift over time
- Parallel run conflicts

Small teams, early-stage projects with low CI volume

Docker / Compose Ephemeral Env

✓ PROS

- Fresh state per run
- Identical dev ↔ CI
- No host-port conflicts (service-name networking)

✕ CONS

- Docker daemon required in CI
- Slower than in-memory

Standard choice for integration and API test layers

Testcontainers (Programmatic)

✓ PROS

- Controlled from test code
- Auto-cleanup via Ryuk
- Parallel-safe — no fixed ports

✕ CONS

- Requires Docker-API runtime
- More complex for first-timers

Integration tests needing real DB / broker / service dependencies

Ephemeral Cloud / PR Environments

✓ PROS

- Production-like isolation
- Per-PR preview environments
- Zero infra management

✕ CONS

- Slower provisioning
- Cost per environment

Staging-level E2E tests; environment-as-a-service platforms

Dockerfile for Test Execution

Dockerfile.test — Maven + Java 21 (temurin-alpine)

```
# Dockerfile.test — test execution image
FROM eclipse-temurin:21-jdk-alpine

# Install Maven (or use Maven wrapper in repo)
RUN apk add --no-cache maven

WORKDIR /app

# Copy project files
COPY pom.xml .
COPY src/ src/

# Pre-fetch dependencies (cached layer)
RUN mvn dependency:go-offline -B

# Run tests by default
CMD ["mvn", "test", "-B", "--no-transfer-progress"]

# docker-compose.test.yml
# Passwords via env-vars — never hardcode in file
services:
  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: testdb
      POSTGRES_USER: testuser
      POSTGRES_PASSWORD: ${DB_PASS}  # from CI secret
    healthcheck:
      test: ["CMD","pg_isready","-U","testuser"]
      interval: 5s
      retries: 5

  test-runner:
    build:
      context: .
      dockerfile: Dockerfile.test
    depends_on:
      postgres:
        condition: service_healthy
    environment:
      DB_URL: jdbc:postgresql://postgres:5432/testdb
      DB_USER: testuser
      DB_PASS: ${DB_PASS}  # injected from CI secret
```

docker-compose.test.yml — env-var injection (no hardcoded secrets)

Docker Test Execution — Key Rules

Use Alpine / slim base images

eclipse-temurin:21-jdk-alpine is ~200MB vs ~500MB for full JDK. Smaller images = faster CI layer pulls. Pre-fetch dependencies in a separate RUN layer for effective caching.

Health checks before test runner starts

depends_on: condition: service_healthy ensures the database is accepting connections before tests run. Without this, tests fail with connection-refused errors on slow CI runners.

Never use fixed ports — use Docker networking

Container-to-container communication uses service names (postgres:5432), not localhost. This avoids port collisions in parallel CI runs and mirrors production networking.

Docker-in-Docker (DinD) — avoid where possible

DinD is complex, needs privileged mode, and is prone to performance issues. Prefer running tests in a container with Docker socket mounted, or use Testcontainers Cloud.

Never commit credentials to Dockerfile

Pass DB passwords, API keys, and secrets via CI env-vars (environment: in compose, or --env in docker run). Store secrets in your CI secrets store, not in the image.

Testcontainers — Programmatic Docker in Tests

Testcontainers: open-source library providing a programmatic API to spin up Docker containers within test lifecycle hooks. Supports Java, Python, Go, .NET, Node.js. Auto-cleans containers via Ryuk sidecar. Requires a Docker-API compatible runtime — no DinD needed.

Testcontainers — PostgreSQL integration test

```
// Maven dependency
<dependency>
  <groupId>org.testcontainers</groupId>
  <artifactId>postgresql</artifactId>
  <scope>test</scope>
</dependency>

// JUnit 5 integration test
@Testcontainers
class UserRepositoryIT {

  @Container
  static PostgreSQLContainer<?> postgres =
    new PostgreSQLContainer<>("postgres:16-alpine")
      .withDatabaseName("testdb")
      .withUsername("testuser")
      .withPassword("testpass");

  @BeforeEach
  void setUp() {
    // Testcontainers maps to a RANDOM port
    // — get it via postgres.getMappedPort(5432)
    DataSource ds = DataSourceBuilder.create()
      .url(postgres.getJdbcUrl())
      .username(postgres.getUsername())
      .password(postgres.getPassword())
      .build();
  }

  @Test
  void saveUser_persistsToRealDatabase() {
    // test runs against REAL postgres — not H2
  }
}
```

Testcontainers Key Concepts

@Testcontainers + @Container

@Testcontainers enables JUnit 5 extension. Static @Container is shared across all tests in the class (started once). Non-static = new container per test method.

Random ports — never use fixed ports

Testcontainers maps container ports to random host ports automatically. Use getMappedPort() or getJdbcUrl() to connect. Fixed ports cause collisions in parallel CI runs.

Ryuk auto-cleanup

Testcontainers attaches labels to all created resources. The Ryuk sidecar removes them automatically after the test process exits — even on crash or SIGKILL.

Wait strategies — reliability in CI

Containers start before they are ready. Use .waitingFor(Wait.forHealthcheck()) or .waitingFor(Wait.forLogMessage("ready")) to block until the service is fully initialised.

Testcontainers Cloud — for DinD-free CI

Runs containers remotely; no Docker daemon needed on the CI runner. Eliminates DinD complexity and privileged mode. Useful for GitHub Actions hosted runners.

Test Data Management Strategies

Core Strategies

Seed scripts in version control

Store SQL/JSON seed scripts in src/test/resources/data/. Run before tests via @BeforeAll or init scripts. Every developer and CI run starts from the same known state.

UUID / timestamp-prefixed test data

Create test records with unique prefixes: user_test_<uuid>, order_<timestamp>. Prevents conflicts between parallel test runs and makes cleanup trivial.

Transactional rollback per test

Wrap each test in a @Transactional annotation (Spring) or begin/rollback block. DB is reset after every test without running cleanup scripts — fast and reliable.

Scheduled daily refresh for shared envs

For shared staging environments, run a nightly data-refresh job (CI scheduled workflow) to restore a clean baseline. Prevents drift accumulation over days of testing.

Never use production data — mask or synthesise

Production data in test environments violates GDPR/DPDP. Use synthetic data generators or masked exports. Mask email, phone, PAN fields at minimum before seeding.

src/test/resources/data/seed.sql

```sql
-- src/test/resources/data/seed.sql
-- Run once before test suite via @BeforeAll
DELETE FROM orders WHERE user_id LIKE 'test_%';
DELETE FROM users  WHERE id LIKE 'test_%';

INSERT INTO users (id, name, email) VALUES
  ('test_alice', 'Alice',
   'alice_test@example.com'),
  ('test_bob',   'Bob',
   'bob_test@example.com');

INSERT INTO orders (id, user_id, amount) VALUES
  ('ord_001', 'test_alice', 99.99),
  ('ord_002', 'test_bob',   49.50);
```

application-test.properties — env-var injection

```properties
# src/test/resources/application-test.properties
# Override for test profile — injected via CI env-vars

spring.datasource.url=${DB_URL:jdbc:h2:mem:testdb}
spring.datasource.username=${DB_USER:sa}
spring.datasource.password=${DB_PASS:}

# Disable real email/SMS in test env
app.notifications.enabled=false
app.payment.gateway=sandbox

# Never hardcode credentials — use env-vars
# Set DB_URL, DB_USER, DB_PASS in CI secrets
```

Docker Test Execution in CI Pipeline

GitHub Actions — Docker Compose test job

```yaml
integration-test:
  needs: unit-test
  runs-on: ubuntu-latest   # Docker pre-installed
  steps:
    - uses: actions/checkout@v6

    - name: Build test image
      run: |
        docker build -f Dockerfile.test \
          -t myapp-test:${{ github.sha }} .

    - name: Run integration tests
      run: |
        docker compose \
          -f docker-compose.test.yml \
          up --abort-on-container-exit \
          --exit-code-from test-runner
      env:
        DB_PASS: ${{ secrets.TEST_DB_PASS }}

    - name: Copy test reports from container
      if: always()
      run: |
        docker compose \
          -f docker-compose.test.yml \
          cp test-runner:/app/target/surefire-reports \
          ./test-reports

    - name: Tear down containers and volumes
      if: always()
      run: |
        docker compose \
          -f docker-compose.test.yml down -v

    - uses: actions/upload-artifact@v4
      if: always()
      with:
        name: integration-test-results
        path: ./test-reports/
```

CI Integration Notes

ubuntu-latest has Docker pre-installed

GitHub-hosted ubuntu-latest runners include Docker Engine. No setup step needed — you can call docker and docker compose directly.

--abort-on-container-exit

Stops all services as soon as any container exits. --exit-code-from test-runner propagates the test runner exit code to the CI job — pipeline fails if tests fail.

Secrets via CI env-vars, never in compose file

Pass DB_PASS via GitHub secrets → env: block. The compose file reads it as ${DB_PASS}. Never write passwords in docker-compose.test.yml — it lives in version control.

Copy reports before compose down

Reports inside the test-runner container are lost when compose cleans up. Use docker compose cp to extract them before teardown, then upload-artifact.

DEMO

Demo: Containerise → Seed → Run → Report

01 Running Tests in Docker Containers

- Write Dockerfile.test based on eclipse-temurin:21-jdk-alpine
- Build image: docker build -f Dockerfile.test -t myapp-test .
- Run with Docker Compose: docker compose -f docker-compose.test.yml up --abort-on-container-exit
- Observe: postgres health check completes before test-runner starts

02 Configuring Environment Variables for Tests

- Show application-test.properties reading ${DB_URL}, ${DB_USER}, ${DB_PASS}
- Run without env-vars — observe fallback to H2 in-memory (default value)
- Export DB_URL pointing to dockerised postgres — observe switch to real DB
- Push to CI — inject secrets via GitHub repo settings → confirm pipeline uses them

03 Managing Test Data Sets

- Run seed.sql via @BeforeAll — show deterministic test data in place before tests
- Demonstrate UUID-prefixed data creation: INSERT user_test_<uuid> during test
- Show transactional rollback: verify DB state is clean after each test method
- Introduce a data conflict (duplicate key) — observe failure, fix with unique prefix

HANDS-ON LAB

Lab: Containerise, Seed & Configure

40% Theory | 60% Hands-on

Task 1

Docker image running tests (screenshot)

Create Dockerfile for Test Execution

1. Create Dockerfile.test: FROM eclipse-temurin:21-jdk-alpine, COPY pom.xml + src/
2. Add RUN mvn dependency:go-offline for dependency caching layer
3. Build: docker build -f Dockerfile.test -t myapp-test . — verify image builds cleanly
4. Run tests: docker run --rm myapp-test mvn test — confirm tests pass inside container

Task 2

Compose run with green test output

Set Up Docker Compose with Test Database

1. Write docker-compose.test.yml: postgres:16-alpine service with health check
2. Add test-runner service with depends_on: postgres: condition: service_healthy
3. Pass DB_URL via environment: block (read from CI env-var, not hardcoded)
4. Run: docker compose -f docker-compose.test.yml up --abort-on-container-exit

Task 3

Environment config doc + working CI job

Environment Configuration Documentation

1. Create application-test.properties using ${DB_URL}, ${DB_USER}, ${DB_PASS} with H2 fallbacks
2. Write seed.sql with prefixed test records; run via @BeforeAll ScriptUtils or Flyway
3. Add CI job to ci.yml: build image → compose up → copy reports → upload-artifact
4. Document the environment setup: which env-vars are required, where secrets are stored

Day 6 Deliverables & Key Takeaways

Deliverables You Take Away

Dockerfile for test execution + docker-compose.test.yml with health-checked database service

Environment Configuration Documentation — env-vars map, secrets locations, seed data guide

Key Takeaways

1. Docker ephemeral environments eliminate 'works on my machine' — tests run identically on dev and CI
2. Never use fixed ports in Docker test setups — random ports prevent parallel run collisions
3. Testcontainers gives programmatic control of containers in test lifecycle hooks with auto-cleanup via Ryuk
4. Seed scripts in version control + UUID-prefixed data = reproducible, conflict-free test data
5. Never commit credentials to Dockerfiles or compose files — inject via CI secrets and env-vars
6. Never use unmasked production data in test environments — GDPR/DPDP compliance is non-negotiable

Tomorrow — Day 7: Test Reporting, Quality Gates & Optimisation · Allure, coverage gates, pipeline dashboards

Hands-on: Generate Allure reports · configure coverage thresholds · optimise slow CI pipeline

RECAP QUIZ

Module 6 – Quick Knowledge Check

Q1. How do Testcontainers and Docker Compose each handle port assignment, and why does it matter for CI?

- Testcontainers automatically maps container ports to random host ports — use getMappedPort() or getJdbcUrl() to connect. This prevents collisions when parallel test runs start multiple containers on the same host. Docker Compose uses internal service-name networking (postgres:5432 inside the Compose network) — host ports are only published if you explicitly configure them, so avoid fixed published ports unless host access is actually needed.

Q2. What is the role of the Ryuk sidecar in Testcontainers?

- Ryuk is a sidecar container started by Testcontainers that tracks all created resources (containers, volumes, networks) via labels. It automatically removes them after the test process exits — including on crash or SIGKILL — ensuring no orphaned containers accumulate on CI runners.

Q3. A test passes locally against H2 but fails in CI against PostgreSQL. Name two likely causes.

- 1) SQL dialect differences: H2 is lenient about syntax that PostgreSQL enforces strictly (e.g. quoting, type coercions, unsupported functions). 2) Environment config mismatch: the CI DB_URL env-var is pointing to the wrong host, or credentials are incorrect — check the CI secret values and connection string.

Q4. Why is production data prohibited in test environments, and what are the alternatives?

- Production data contains PII (names, emails, payment info) protected under GDPR, DPDP, and HIPAA. Using it untreated in test environments risks legal liability and data leakage. Alternatives: synthetic data generators, masked/anonymised exports (email → anonymised, phone → redacted), or seed scripts with realistic but fictional test records.